

ThemeManager Deep-Dive

Overview

ThemeManager is the singleton responsible for applying and hot-switching the application's visual theme. It centralises colour, font and UI defaults so the rest of the GUI framework can remain brand-agnostic.

Responsibility	Details
Apply initial theme	Called during MainFrame bootstrap to patch UIManager defaults.
Live theme switch	Offers setTheme(String name) to switch palettes at runtime and repaint all registered roots.
Component refresh	Maintains a list of root components (JComponent) to recursively update when theme changes.

Key Collaborators

- **ThemeColors** – supplies the colour palette.
- **ThemeFonts** – (optional) supplies font sizes/weights.
- **Themelcons** – relies on current palette for fallback icon generation.
- *Styled components** – query **ThemeColors** after Manager applies palette.

Public API

```
ThemeManager mgr = ThemeManager.getInstance();
mgr.setTheme("dark"); // switch to dark mode at runtime
mgr.registerRoot(mainFrame); // make sure MainFrame repaints when theme
flips
```

Internal Workflow Sequence

Below PlantUML shows what happens when **setTheme()** is invoked.

```
@startuml
actor User
User -> ThemeManager : setTheme("dark")
ThemeManager -> ThemeColors : loadPalette("dark")
ThemeManager -> UIManager : put(all swing defaults)
ThemeManager -> RegisteredRoot* : updateUI()
@enduml
```

Design Rationale

- **Singleton** – ensures a single authoritative theme across the app; avoids inconsistent palettes.
- **Observer-like refresh** – instead of Swing's LAF change, we manually track roots; lighter and avoids full LAF reload.
- **Separation of palette** – **ThemeColors** can be swapped without touching Manager logic.

Code Snippets

Registering a new root component

```
public class BasePanel extends JPanel {  
    public BasePanel() {  
        ThemeManager.getInstance().registerRoot(this);  
    }  
}
```

Adding a new theme

```
// ThemeManager.setTheme(String)  
case "solarized":  
    ThemeColors.loadSolarized();  
    break;
```

Future Extensions

- Persist last-selected theme in user preferences.
- Expose **PropertyChangeSupport** so components can listen to specific theme property changes.